

Chapter 1

Introduction

Modern software is often written in such a way that it is event-driven, or in other words, **reactive**. This means that applications react to an incoming event by creating subsequent events that arise as a reaction to the incoming event. Such an application is often called a **reactive system**. Reactive systems are often implemented with callbacks or by implementing the Observer Pattern. However, this does not come without its drawbacks as this may lead to 'inversion of control' or 'callback hell' [1][2]. To alleviate this we introduce **Reactive Programming**. A programming paradigm that allows the user to write reactive systems in a declarative way. Traditionally the sum and the product of two elements would be written as shown in Listing 1.

```
int x = 5;
int y = 10;
int s = x + y;
int p = x * y;
// x changes state
x = 10;
// s requires recomputation
s = x + y;
// p requires recomputation
p = x * y;
```

Listing 1: Imperative product and sum

Whenever x or y would change state, you would have to manually update s and p . Reactive programming allows you to declare x and y as sources of data and s and p as the result of the relevant computations. Whenever the value of x or y would change, the variables dependent on the sources would be automatically recomputed, and the result propagated further in the reactive system. Schematically the dependency graph of product-sum would look like Figure 1.1

Toon eerst de code (in een taal of pseudo code, elk zijn voor- en nadeel)

"we introduce" is lingo voor "dit hebben wij gemaakt".

RP is niet jouw uitvinding? (ook niet de mijne).

Als je bedoelt dat je eerst iets introduceert, zeg dat dan. maar hier best het 100% als "dit is van ons/ing".

voorbeeldje? (Dit kan tekstueel zijn = uitleg, nu is het een "name drop")

dit uitleggen wáár dat events vandaan kunnen komen én dan pas dat de verwerking van een event inductief vordert dan veroorzaken.

wat is "this"? (waar verugst dit naar?).

En dat is? (nog een name drop).

Afgewen bespuitelijk een "generieke (CS-professor)". Ann Nowé / Gerard Wiggans kennen dit niet!

Ah, "traditioneel" is "Imperatief Programmeren". zeg dat dan hier.

Als je dit wil doen maak dan update-s() en update-p() als functies ipv expressies te gebruiken

Meld: printbaarheid is hier vreemd. PDF is ok.

2

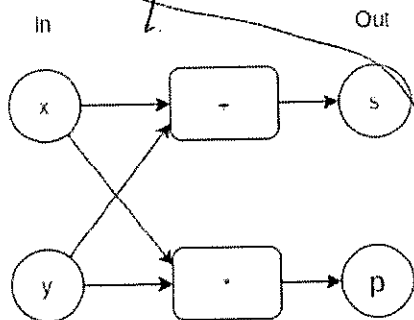


Figure 1.1: Dependency graph for product-sum

There are different ways of how such a reactive system could be created using Reactive Programming. One way is to use a programming which itself is natively reactive. A few examples are Haai [3]. Another is to add native reactivity to an existing language by extending it the language or by implementing a reactive library such as ReactiveX. We will be looking at the former and more specifically the higher-order reactive programming language Haai, which was developed at the SOFT lab at the VUB. Haai uses reactors as their As a preview, an example of how such a product and sum could be written in Haai is shown in Listing 2. Haai uses reactors as its native way of declaring a reactive program.

```
(dofr (product-sum x y)
  (def sum (+ x y))
  (def product (* x y))
  (out product sum))
```

Examples
(= plural),
followed
by one?

the only computational
abstraction for

Listing 2: Product sum in Haai

Additionally, we want to delve into the world of electronics. From maintaining your fridge's temperature to controlling a drones' altitude, electronics are deeply embedded in everyone's day-to-day life. Writing software for these electronics can also be challenging. This mainly due to the fact that using peripherals require using their associated libraries, which are not always standardised. Another challenge is that creating programs for microcontrollers often require the developer to use a low-level programming language such as C/C++ or Rust, which come with their own challenges. Nowadays, there are some efforts being made to provide a standardised collection of tools to aid with the development of systems with many peripherals, or robots. There is Robot Operating Systems (ROS) [4], Open Robot Control Software (OROCOS) [5], Yet Another Robot Platform (YARP) [6] to name a few. We will be focussing on using (ROS).

1.1 Contributions

Reactive programming is used across a multitude of domains. It is used in web and mobile development for instance. However, we will be focussing on using it for the Internet of Things

→ wát doen deze tools?
(Abstrakt). Niet duidelijk
waar ze passen in de ontwikkelcyclus.

2. opkleven geen
motivatie

been there is overige za
we willen veel willen...

Andere motivatie nodig.
One popular application domain
for event-based ...
= robotijk-here overgang doen o.k.

Tip: oplossen in PDF,
include graphics? dan PDF's
includen.
CHAPTER 1. INTRODUCTION
Vectoriële
= geen pixels,
(ik denk de voorwaart,
op den printer, van)

Strikt genomen
geen RP-tool
Meer reactiv
Couria
(zie 2013
Survey
Paper).
Maar
ook
somt ijs
heerlijk
verrie)

why? ~~dit is~~ Dit is
geen geldige reden.
Motiveer d.t.

Devices in

1.2. OVERVIEW OF THIS DISSERTATION

3

(IoT). IoT consists of multiple physical objects, usually with very limited computing power and/or memory. In this thesis we introduce a way of declaring a reactive program across multiple devices from a single source. This means that the developer declares the program once, along with declarations where which part of the program lives and then deploys the program to the relevant devices making use of ROS. This is achieved by the following components:

- An extension of the Remus VM [7], more specifically, extending the language to accept a declaration of where which part of the program lives. The Remus compiler will use this metadata to split the Haai reactor declarations into an embedded part and a server part. Each piece will then be available for the user to be deployed in their respective parts of the IoT network.
- A new tool called CHaai, which translates the Remus bytecode obtained from compiling the Haai program into a C program. The C program will be required to run the program on microcontrollers using ROS.
- A C port of the existing Remus VM written in Rust [8], also created to allow developers to create reactive programs for microcontrollers. However, it is adapted with respects to the current Instruction Set Architecture (ISA) of the Racket version of Remus and allows usage of ROS.

1.2 Overview of this Dissertation

Chapter 2 gives the problem statement as well as an overview of the current reactive programming solutions and existing toolkits to program networks of microcontrollers. It also introduces a concept called 'Tierless programming', which will inspire the declarative way of telling Remus where which part of the reactive program lives.

Chapter 3 introduces μ -Remus. A C port of the Remus VM, which was previously written in Rust as well as in Racket. It explains why the C port was necessary as well as conceptualizes the link between the ROS communication protocols and the dependency graph generated by reactive programming.

Chapter 4 demonstrates usage of the Haai language extensions inspired by Tierless programming to declare where which part of the reactive program lives. It will also tell you about the CHaai tool. Which will create a C version of the reactive program. It is ported to C to be compatible with ROS.

Chapter 5 will evaluate the tools and extensions driven by an example. The target microcontroller will be a Raspberry Pi Pico H (2021) and will be used along a PC.

Lastly, chapter 6 will conclude this dissertation and give notes on some possible future work.

"Background" / "State-of-the-Art" als begrip(per)
vermoeden -

veel verwinnen hier.
ik vermoed een LaTeX-
dingetje.

Waarom zijn beide nodig? Motivatie in onderdelen

Op de punt in de tekst.

Wat dit gaan de
Problem statement
en/of approach (zie p.4).
Motivatie waarom deze
contributies precies
nodig zijn en (breuk)

moet er een al toegevoegd zijn maar
hier ga je er uiteraard uitgebreid
op in.

ROS is nog niet toegevoegd in het huidige
verhaal.

Geen thesis eigo. Een thesis
heeft niet
problemen...

Dit is geen approach - sectie.
 Approach als sectie in watvoor
 de contributies hoort.
 4
 CHAPTER 1. INTRODUCTION
 1.3 Approach

We first start by discussing the state of the art, what tools already exist and then move on to the problem statement and continue with the contributions part of the dissertation. However, since it is heavily dependent on a language and a toolkit we will first give a brief introduction into some relevant parts of the language or toolkit, each time providing the necessary context and information, prior to moving on to the actual contributions.

Heb je
 al
 gezien

p. 3-

Dus

1. Introduction

1.1. Problem Statement

1.2. Approach.

1.3 Contributions.

1.4 Structure of the Thesis.

3 en een beetje pagina is scherp voor een introductie-
 hoofdstuk voor een master thesis. Geef een duiding?